2023BCSE008

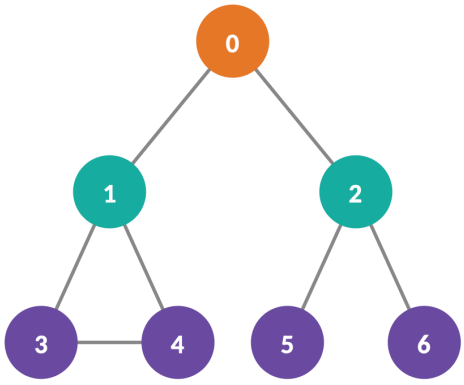
LAB : 04 DATE: 2|04|2026

Lab 4

Artificial Intelligence Lab

1 2 3 4

The Graph You Will Use for All Programs



Edges: (0,1) (0,2) (1,3) (1,4) (2,5) (2,6) (3,4)

Adjacency List (Undirected Graph)

Node 0	→	**1, 2**
Node 1	→	0, 3, 4
Node 2	→	0, 5, 6
Node 3	→	1, 4
Node 4	→	1, 3
Node 5	→	2
Node 6	→	2

Program 1 DFS Traversal & Tree on a Given Graph

Perform DFS on the given graph starting from Node 0. Print the order in which nodes are visited. Also show the DFS tree by printing each node and its parent (e.g. 0 → 1 → 3).

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lan_04 > dfs.py > ...

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
def dfs(node, visited):
    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor, visited)

visited = set()
print("DFS Order:")
dfs(0, visited)
```

OUTPUT:—

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desk
gar/AI_Lan_04/dfs.py"
DFS Order:
0 1 3 4 2 5 6 %
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

2023BCSE008

Program 2 **Find a Path Between Two Nodes**

Using DFS on the given graph, find if a path exists from Node 0 to Node 6. Print the complete path if found (e.g. 0 → 2 → 5 → 6), or print 'No path found'. Stop as soon as the destination is reached.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lan_04 > path.py > ...

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
def dfs_path(node, dest, visited, path):
    visited.add(node)
    path.append(node)

    if node == dest:
        print("Path found:", " → ".join(map(str, path)))
        return True

    for neighbor in graph[node]:
        if neighbor not in visited:
            if dfs_path(neighbor, dest, visited, path):
                return True

    # Backtracking
    path.pop()
    return False
visited = set()
source = 0
destination = 6

if not dfs_path(source, destination, visited, []):
    print("No path found")
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -
gar/AI_Lan_04/path.py"
Path found: 0 → 2 → 6
```

2023BCSE008

Program 3 **Detect a Cycle in the Graph**

Check whether the given graph contains a cycle using DFS. Print 'Cycle detected' or 'No cycle found'. Also test your program on a second graph (a simple tree) to verify the 'No cycle' case.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lan_04 > cycle.py > dfs_cy

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
def dfs_cycle(node, visited, parent):
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            if dfs_cycle(neighbor, visited, node):
                return True
        elif neighbor != parent:
            return True

    return False

visited = set()

if dfs_cycle(0, visited, -1):
    print("Cycle detected")
else:
    print("No cycle found")
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligenc
gar/AI_Lan_04/cycle.py"
Cycle detected
gauravmathur@Annus-Mac Artificial_Intelligenc
```

2023BCSE008

Program 4 **Topological Sort (Directed Version)**

Convert the given graph into a Directed Acyclic Graph (DAG) by making all edges point from lower to higher node numbers. Perform topological sorting using DFS and print the result.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lan_04 > topological.py > ...

```python
dag = {
    0: [1, 2],
    1: [3, 4],
    2: [5, 6],
    3: [4],
    4: [],
    5: [],
    6: []
}
def dfs_topo(node, visited, stack):
    visited.add(node)

    for neighbor in dag[node]:
        if neighbor not in visited:
            dfs_topo(neighbor, visited, stack)

    stack.append(node)

visited = set()
stack = []

for node in dag:
    if node not in visited:
        dfs_topo(node, visited, stack)

print("Topological Order:")
print(stack[::-1])   # reverse

```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Us
gar/AI_Lan_04/topological.py"
Topological Order:
[0, 2, 6, 5, 1, 3, 4]
```